MOAI RESEARCH · OCTOBER 2026

Ideas for ObjectFlow and ManMap

What SAP RAP, Business Central, the Java model-driven platforms, the open-source ERPs, the persistence ecosystem, event and process frameworks, modern type systems and low-code products declare that MoWare applications still build by hand, and what the two languages could absorb.

Compiled from seven research notes in `research/research_notes/Ideas for ObjectFlow and ManMap/`. Source: `research/reports/Ideas for ObjectFlow and ManMap.md`.

Für WM,
der nie aufhört, nach neuen Ideen zu suchen.

“The best way to have a good idea is to have a lot of ideas.”

Linus Pauling

Lifting Lifecycle, Events and Graph Persistence into the MoWare DSLs

Across SAP RAP, Business Central, the Java model-driven platforms, the open-source ERPs, the Java persistence ecosystem, the event and process frameworks, modern type systems and the low-code products, the same handful of concept families recur as *declarations* that MoWare applications still build by hand: rules on the domain model with declared triggers and a fixed save phase, a transition table on the Status concept, domain events with a guaranteed post-commit dispatch, a persisted process or work queue with retry and deadline semantics, a declared composition that generates parent-then-children save and orphan delete, read models whose SELECT is derived from a declared shape, and per-property facts (nullability, constraints, dimensioned numbers) that one declaration feeds into generated Java, the schema, serdes and any API contract. Three runtime capabilities carry most of these ideas: the Session as a *phased* unit of work with named hook points (it already owns the transaction, so determinations, validations, event registry rows and outbox rows can be generated into the check-in), a *persisted scheduler and work queue* (deadlines, timers, retries, incidents, idempotency keys and process instances all need it, and the volatile batch inbox is the only thing in that space today), and a *graph-aware generator* for ManMap (compositions, load graphs, derived views, aggregates and historisation are all mechanical once the FK graph is declared). The external evidence also validates several deliberate MoWare omissions, notably no lazy loading, no implicit cascade and no event sourcing, so the recommendation is to add declared, visible, non-transitive variants of these mechanisms rather than copying ORM defaults. The ideas below are grouped by theme; every idea names the external systems that do it, sketches the ObjectFlow or ManMap form, and states briefly how MoWare developers hand-build it today, drawing on `docu/objectflow.md` (OF), `docu/manmap.md` (MM), `docu/moware-werkbank.md` (WB), `docu/dataux.md` (DUX), `conventions/` (CONV) and the two problem logs (PR1, PR2) as summarized in the baseline note. A separate chapter maps every recurring hand-written pattern to the ideas that would absorb it, and a final chapter lists the small options and checker rules.

How to read the evidence

The notes rest mostly on primary documentation (vendor manuals, specifications, source code, JEPs), but several areas rest on secondary material and should be weighed accordingly. SAP Help Portal and SAP Community pages were unreachable, so RAP facts come from SAP's own Learning course, GitHub cheat sheets, developer tutorials and SAP Press; the RAP background-processing framework (bgPF) and RAP change documents rest on search excerpts only, and the claim that managed RAP "handles change documents automatically" (practitioner blog) is contradicted by a SAP Community answer saying an additional save plus an API call is required, so change documents should be treated as hand-written in RAP (SAP cheat sheet 08). Salesforce's developer and help sites were unreachable; its order of execution is reproduced from a secondary blog and a search summary that disagree on where custom validation rules sit relative to before-save flows (salesforcegeek). iDempiere wiki pages were blocked, so several iDempiere facts are search snippets, while the code-level facts come from the GitHub source. Odoo's workflow-engine removal (v10/v11) is sourced only from forum threads, and Odoo 19's release month is given as September by one source and October by another. Hibernate's "no API to include soft-deleted rows" rests on a forum thread. Flyway naming conventions come from third-party guides because the official page returned 404. Fowler's `allocate` for Money rests on secondary write-ups of PoEAA. The Camunda-versus-code-first comparison is vendor and consultant material. Finally, every BIG/SMALL classification in the notes is the researchers' judgment, not a sourced fact; this report uses it as a proposal.

Theme A. Rules and types on the domain model

A1. Declared constraints and refinement types on Business Properties

The idea is to let a Business Property or a named single-property value type carry its constraint (length range, numeric range, scale, pattern, non-empty, unique) so that one declaration produces a validated constructor, the schema (`VARCHAR(n)`, `DECIMAL(p,s)`, `CHECK`, `UNIQUE`, `NOT NULL`), serdes and API-contract keywords, and UI metadata. Typed languages do this as refinement types: Iron for Scala 3 writes `Int :| Positive` or `String :| Alphanumeric`, checks literals at compile time and refines runtime values through `refineEither`/`refineValidation` (the latter accumulating all errors), and reuses the constraint in JSON, database and config integrations (Iron). Kotlin value classes carry an `init { require(...) }` block so the type can only be constructed through one validated path (Kotlin inline classes); Rust's book shows the same with a private-field `Guess` and a checking constructor so downstream functions "wouldn't need to do any additional checks" (Rust book ch09-03); F# single-case unions wrap primitives to prevent mixing an email with a zip code (F# single-case DUs). Alexis King's "parse, don't validate" names the design rule: a check should return a more precise type so the proof is kept, and should happen once at the boundary rather than as "shotgun parsing" spread through processing code (Parse, don't validate). The low-code platforms do the same declaratively: Mendix entity validation rules are a closed list (required, unique, equals, range, regular expression, maximum length) checked on commit in configured order with uniqueness last because it needs the database, and all failing messages are recorded and surfaced per caller (page message, microflow error or Java exception) (Mendix validation rules); Salesforce validation rules are a true/false formula plus a message anchored to a specific field, with `ISNEW()`, `ISCHANGED(field)` and `PRIORVALUE(field)` to scope a rule to a transition (Trailhead validation rules, best practices). Causeway's `@Property` carries `maxLength`, `regexPattern`, `optionality` and `mustSatisfy` specifications, and a mismatch between `optionality` and `@Column(nullable)` fails at boot (Causeway @Property). OpenAPI 3.1 and JSON Schema carry `minLength`, `pattern`, `minimum` and `exclusiveMinimum` in the contract (OpenAPI 3.0 to 3.1).

In ObjectFlow this would read as constraint clauses on the property type, for example `string(1..60)`, `BigDecimal(12,2) >= 0`, `pattern '...'`, plus a `value type CustomerNo = string(8) pattern '[0-9]{8}'` root concept usable as a property type in Entities, Value Objects and DTOs and mapped by ManMap to the underlying column type without a schema change. Generated Value Objects and value types would be Java records whose compact constructor hosts the check (JEP 395); a literal assigned to a constrained property could be checked in the MPS editor, mirroring Iron's compile-time literal check. The DTO-to-Entity boundary is where the one conversion raises user-correctable Preconditions, so Entity code downstream carries no re-checks.

In MoWare today. Property options already include `LENGTH` (min/max, which also drives mandatory input in the UI) and `RANGE` (int and BigDecimal), and ManMap adds `SIZE`, `NOTNULL`, `UNIQUE` as schema hints (OF § "Business Properties"; MM § "Felder, Schlüssel und Optionen"). Everything beyond that lives in Preconditions inside Service Methods, repeated per use case and invisible to ManMap, serdes and any API. The conventions require DTO input checks as CheapCode that "never replace service checks", and PR1 P-033 records that validations were modeled twice because `LENGTH` already controlled mandatory input (CONV `mw-domaenenmodell_v1.md` § "Eingabeprüfungen und fachliche Prüfungen"; PR1 P-033). A single-property business type such as an invoice number is a bare `string`, so nothing stops a customer number being passed where an invoice number is expected.

A2. Entity rules with declared triggers and a phased save

Both large commercial platforms attach rules to the entity and declare *when* they run rather than where they are called. SAP RAP distinguishes determinations (derive values) from validations (reject), each with trigger conditions `create; update; delete; field a, b;` and a phase: `on modify` runs immediately in the transactional buffer and may fire several times, `on save` runs exactly once in the save sequence (`finalize` for determinations, `check_before_save` for validations); a failed validation "rejects the entire buffer"; execution order among determinations and validations is "not fixed" (SAP cheat sheet 36, SAP Learning determinations, SAP Learning validations). SAP Press frames the on-modify versus on-save choice as "dependency, not cost" and warns that a field used as trigger fires whenever that field changes (SAP Press

decision model); practitioner blogs report that teams "put too much logic in determinations, making them behave like mini workflows" and that mixing determinations and validations up "causes subtle bugs" (dev.to RAP deep dive). RAP's save sequence names the phases (`finalize`, `check_before_save`, `adjust_numbers`, `save`, `cleanup`) with a point of no return, and strict mode turns phase violations (database writes outside the late save, events raised outside save) into checks (SAP cheat sheet 08). Business Central puts the same on the table: `OnInsert`/`OnModify`/`OnDelete` run before the database operation and cancel it on error, per-field `OnValidate` runs after type and property checks, and the documented order around a delete is before-event, table trigger, global trigger, database operation, after-event; the catch is that triggers run only when the caller passes `RunTrigger = true`, a known source of silently skipped logic (BC OnModify, BC OnValidate, BC event types). The ERP family converges on the same vocabulary: iDempiere's `ModelValidator` enumerates `TYPE_BEFORE_NEW`/`CHANGE`/`DELETE` and `AFTER_*` plus sixteen document timings (ModelValidator.java); metasfresh's `@ModelChange(types, ifColumnsChanged, afterCommit)` makes the hook column-aware and lets it run after commit (ModelChange.java); Odoo's `@api.constrains('name','description')` runs only when a listed field is in the written values (Odoo ORM); Mendix event handlers are Before/After x Create/Commit/Delete microflows returning a Boolean, with "raise an error when the microflow returns false" making a Before Commit handler behave like native validation (Mendix event handlers); Jmix's `EntityChangedEvent` carries `AttributeChanges` with old values and distinguishes a before-commit listener (exception rolls everything back) from an after-commit one (Jmix entity events); Salesforce fixes one documented save pipeline from validation rules through before triggers, save, after triggers, roll-ups and commit, with the warning that workflow field updates re-fire triggers but not validation (salesforcegeek, secondary).

In ObjectFlow the natural form is a rule family declared on an Entity: `determination setDueDate on modify { create; field invoiceDate, paymentTerms }` and `validation datesConsistent on save { create; update; field beginDate, endDate }`, with the generator deriving the method stubs and the Session running them at check-in in a fixed, named phase order: determinations, validations collecting all violations into one problem report, then `save with` statements, then post-commit reactions (see B3). The rule body stays imperative BaseLanguage with access to the prior value and per-property change flags. The checker gains cheap, high-value rules borrowed from RAP's strict mode: no ManMap `CHECKIN`/`DELETE` or `STATEMENT` call inside a determination or validation, no event emission before commit, and cycle detection over field triggers so determinations cannot feed each other indefinitely.

In MoWare today. Preconditions belong to a Service Method and run when that method is called; the recommended pattern is "load facts, one `validation` block, then mutate", and the language does not enforce that checks happen before changes or at all (OF § "Preconditions, Validation, Guards und Exceptions"; WB § "Grundprinzipien" item 6). Nothing in the language ties a rule to an Entity or to a change of a property; a Service that forgets to call the check simply persists the inconsistent graph, because "Properties einer Entity sind auch von Commands und anderen Services aus setzbar" (CONV `mw-domaenenmodell_v1.md` § "Service und Aggregat"). Dirty tracking exists per Entity but exposes no per-property old value to business code (OF § "Dirty-Tracking, Read-only und unveränderliche Werte").

A3. Per-property nullability as a declared fact

Kotlin makes absence a type: `String` cannot hold null, `String?` can, and the compiler forces `?.`, `?:` or a smart cast; the documented remaining NPE sources are `!!`, initialization leaks and Java interop with "platform types" of unspecified nullness (Kotlin null safety). Rust has no null; `Option<T>` and `T` are different types, so `x + y` with `y: Option<i8>` fails to compile (Rust book ch06-01). For generated Java, JSpecify 1.0 (July 2024, four annotations, promised never to change incompatibly) makes `@NonNull String` a subtype of `@Nullable String`, with `@NullMarked` on a package making unannotated types non-null and `@NullUnmarked` as the incremental-adoption escape (JSpecify user guide, release). Oracle's own API note restricts `Optional` to method return types (Optional Javadoc), and Michael Ernst argues `Optional` "does not make your code more correct", merely converting an NPE into a `NoSuchElementException`, preferring `@Nullable` annotations with a static checker (Ernst). In contracts, GraphQL is nullable by default with `!` for non-null and per-level list nullability (GraphQL schema), while OpenAPI 3.1 expresses null as `type: [string, "null"]` (OpenAPI 3.0 to 3.1).

For ObjectFlow this means a `mandatory`/`optional` marker (or `T?`) per Business Property with model-level defaults per model or module, so existing models stay "unmarked" and new ones opt in, exactly Kotlin's platform-type migration path. Generated code carries JSpecify annotations, the `:eq:`/`:ne:` null-safe operators become unnecessary on non-nullable properties, `status switch` needs a `null` arm only on nullable Status properties, ManMap derives `NOT NULL`, and serdes and any API contract derive `required`/`!`. The existing rule "a yes/no is a Status" should be kept so that nullable booleans are not reintroduced.

In MoWare today. Nullability is a set of per-type conventions: `string` is never null (SQL `NULL` maps to `""`), `int` starts at 0, `LocalDate`, `DateTime`, Value Object and references start at `null`, and a Status starts at its `ON_CREATION` element; equality operators differ per type, `==` on a Value Object key is a checker error and `LocalDate == LocalDate` silently compiled to a reference comparison (OF § "Null-Werte in Datenstrukturen"; PR1 P-023). A Status that is legitimately "not yet set" needs `ALLOW_NULL_PERSISTANCE` and a constructor that resets the `ON_CREATION` default to `null` (OF § "Status"; PR2 P-011, P-034), and `optional` in ManMap filters treats `0`, `null` and `""` as unset with `-1` and `""` explicitly excluded (MM § "Spezifikum - Filterausdrücke und gemappte Felder").

A4. Status with payload per state

Java 17 and 21 made algebraic data types mainstream: a `sealed interface` with `permits`, records as the product half, and a `switch` that the compiler checks for exhaustiveness, deconstructs record components and throws `MatchException` if a new permitted type appears at runtime; the JEP advises omitting `default` on enum switches so new constants are flagged, and treats `case null` as an explicit arm (JEP 409, JEP 441, JEP 440). Scala 3 enums mix plain values with parameterized cases and warn on non-exhaustive matches (Scala 3 ADTs, Scala tour); Rust enum variants carry different data (`V4(u8,u8,u8)` vs `V6(String)`) (Rust book ch06-01); the F# essay on making illegal states unrepresentable replaces "email optional, post optional" with `EmailOnly | PostOnly | EmailAndPost` so the invalid "neither" case cannot be constructed (F# illegal states). On the wire these are GraphQL unions and OpenAPI `oneOf` with a `discriminator` (OAS latest). Business Central's extensible enums add a strategy flavour: an enum may `implements` an interface and bind each value to a codeunit, so `iAddressProvider := sendTo` dispatches on the value (BC interfaces).

ObjectFlow's Status is already the sum half; the addition is payload per element, for example `status Shipment { Planned; Shipped(carrier: Carrier, trackingNo: string); Delivered(at: DateTime, signedBy: string) }`, generated as a sealed interface plus records with a deconstructing, default-free `status switch`. Fields meaningful in one state only stop being nullable columns with an implicit invariant. ManMap then has two mechanical mappings: one Status column plus nullable per-state columns with a generated `CHECK`, or a per-state side table. Binding each Status element to a Service implementation (BC's `implements`) is a smaller follow-on.

In MoWare today. A Status is a rich enum with persistence value, descriptions, colour and an exhaustive `status switch` (OF § "Status"; § "Statuswerte mit `status switch` behandeln"), but state-specific data lives in ordinary nullable properties of the Entity whose validity depends on the current Status by convention; the generator emits a completeness check only when `default` is absent, and a `null` Status needs a `default: // and null` arm.

A5. Money and quantity as dimensioned types

Fowler's position is that "money isn't actually a first class data type in any mainstream programming language", that a Money class catches adding different currencies, centralizes rounding, must never use real numbers, and needs an `allocate` operation that splits an amount by proportions without losing cents (Fowler, Money; allocation via secondary write-ups Verraes); his Quantity draft generalizes to amount plus unit with rejection of nonsensical additions and explicit conversion, and notes that currency conversion needs a time reference so automatic conversion is usually disallowed (Fowler, Quantity). F# units of measure give compile-time dimension checking (`float<m/s>` cannot be added to `float<m>`) but are erased at runtime, so units cannot be printed (F# units of measure); Kotlin value classes give a distinct type at zero cost except where boxed (Kotlin inline classes). Causeway offers a framework-level value-type contract (parse, render, defaults, decompose) and composite values persisted as `@Embeddable` (Causeway value types).

For ObjectFlow two distinct types are warranted rather than one generic `quantity<unit>`: a `money` type carrying amount and currency at runtime (the F# erasure does not apply because the currency comes from the database) with same-currency add/compare, scalar multiply, `allocate`, and explicit conversion through a dated rate object; and a `quantity<unit>` for physical units. ManMap gets a canonical two-column mapping (amount `DECIMAL` plus currency `CHAR(3)`), with an option to map amount-only when the owning Entity declares a fixed currency context, the storage choice Fowler leaves open.

In MoWare today. Money is a bare `BigDecimal` with a format string and `13.44bd` literals; `double` and `float` are forbidden (OF § "Literale für Datum, Zeitpunkt und Dezimalzahl"). Currency, rounding and proportional distribution are hand-written per application, and nothing prevents adding a net amount to a percentage.

A6. Named, composable specifications and decision tables

Evans and Fowler define a Specification as a predicate object with `isSatisfiedBy` that separates "how to match a candidate" from the candidate, serving selection, validation and building-to-order at once; composite specifications let programmers supply leaves and combinators, and `remainderUnsatisfiedBy` can tell the user why a specification was not met (Evans & Fowler, Specifications). Spring Data's `Specification` composes with `and`/`or` and, since 4.0, reuses the same predicate for select, update and delete (Spring Data Specifications). Odoo domains are one expression language used for search, record rules, automation triggers and window actions, evaluable both in SQL and in memory via `filtered_domain` (Odoo ORM); metasfresh's `IQueryFilter` likewise accepts a model in memory or renders to SQL (IQueryBuilder.java). For tabular rules, DMN decision tables fix multiple-match semantics with hit policies Unique, Any, First, Rule order and Collect with SUM/MIN/MAX/COUNT, where Unique makes several matches an error (Camunda hit policies). Easy Rules is in maintenance mode and Drools lives under the incubating Apache KIE, which argues for a table *expression* in the DSL rather than an embedded engine (Easy Rules, Apache KIE).

In ObjectFlow a `specification OverdueInvoice on Rechnung { ... }` root with `and`/`or`/`not` composition would serve as a Precondition (`precondition OverdueInvoice(rechnung)`), as a filter in a mapped `where` (compiled to SQL where the predicate is expressible over mapped fields), and as an explanation source for the user-correctable message. A `decision table` expression with inputs, outputs and a hit policy enum covers "derive discount, priority or approver from several attributes", with Unique checked at test time.

In MoWare today. A Precondition is bound to one Service Method and expressed once per method; the same business predicate reappears as a ManMap `where` expression or SQL fragment in a `<Thema>LeseRepo`, and as `#Meta` enablement logic in a Scope function (OF § "Preconditions, Validation, Guards und Exceptions"; MM § "Gemappte Abfragen mit `QueryFromMap`"; OF § "Scopes"). Multi-attribute derivations are nested `if` statements in Service Methods.

Theme B. Lifecycle, status and process

B1. A declared transition table on Status

No surveyed business platform has a declarative state machine, which makes this genuinely novel ground. RAP derives permitted operations from status in an imperative `FOR INSTANCE FEATURES` handler (`%features-%update = COND #( WHEN OverallStatus = accepted THEN fc-o-disabled ...)`) and encodes transitions as actions (RAP100 dynamic feature control); Business Central uses a Status enum with Release/Reopen codeunits and a table-driven workflow engine needing about eight registration subscribers (BC status, BC workflow walkthrough); iDempiere computes valid actions per status in a roughly 350-line hand-coded case analysis per table (DocumentEngine.java); Axelor guards transition buttons with `showIf="statusSelect == 1"` in view XML (AOS SaleOrder views); Causeway, OpenXava and Jmix have no construct at all. OFBiz comes closest with `StatusValidChange(statusId, statusIdTo, conditionExpression, transitionName)` rows that services check before changing status (OFBiz entitymodel.xml, OrderServices.java). The state-machine libraries supply the vocabulary: Stateless4j's `permit`, `permitIf(guard)`, `permitInternal` (no exit/entry), `permitReentry`, `ignore`, `permitDynamic`, `onEntry`/`onExit`, `substateOf` (StateConfiguration.java); Spring

Statemachine's `timerOnce` for timed transitions and `StateMachineRuntimePersister` for persistence (Spring Statemachine); XState's pure named guards composed with `and/or/not`, guarded transition arrays with an unguarded default, and delayed `after` transitions (XState guards, transitions). metasfresh adds a useful assertion: `processEx(document, action, expectedDocStatus)` checks the resulting status after the action (IDocumentBL.java).

In ObjectFlow a `transitions` block on the Status declaration, `from DRAFT on freigeben [guard] -> FREIGEGEBEN`, with optional `entry/exit` actions per element and `substate of` grouping, lets the generator derive the Service Method skeleton and its "status in {sources}" Precondition, the enabled set of transitions for the current status (replacing hand-written `#Meta setElements/setEnabled` logic and `generally enabled` expressions on Commands), a DOT or PlantUML rendering, unreachable-state and dead-end checks, and `run command` test assertions on status paths. Hierarchical states cover "all `IN_PROGRESS.*` can be cancelled"; parallel regions can be skipped, as Stateless4j does.

In MoWare today. The convention is one Service Method per transition, named after the business act, with Preconditions on the current Status (CONV `mw-domaenenmodell_v1.md` § "Aggregate und Invarianten"). The legal-transition set exists only implicitly across those methods, cannot be enumerated for UI or tests, and each Command repeats enablement in `generally enabled` or a Scope function. PR2 P-042 records a status-change mask bound to the real property forcing the Command to remember and restore the old Status around the Service check.

B2. Document lifecycle type and append-only posting

The Compiere family embeds a fixed document vocabulary: `DocAction` statuses Drafted, In Progress, Approved, Completed, Closed, Voided, Reversed and actions Prepare, Complete, Void, Close, Reverse-Correct, Reverse-Accrual, Re-activate, with `prepareIt()` and `completeIt()` returning the new status and sixteen before/after hook timings around them; `DocumentEngine.processIt` re-reads `DocStatus` under a row lock and throws `DocStatusChanged` if another session changed it, and reversal creates a counter-document linked via `Reversal_ID` rather than deleting (DocAction Javadoc, DocumentEngine.java). Period control is a separable constraint: `MPeriod.isOpen(ctx, dateAcct, docBaseType, orgId, forPosting)` decides per document base type, organisation and action-versus-posting (MPeriod.java). metasfresh replaced subclassing with a `DocumentHandler` SPI and a typed `DocStatus` enum with predicate methods (DocumentHandler.java). Business Central structures every transaction as journal lines, a Check codeunit, a Post Line codeunit that is "the only place to insert general ledger ... entries" and immutable ledger entries plus a register inside one transaction with explicit table locks; Microsoft's own account says codeunit 12 had grown to about 7,600 lines before the posting-engine redesign (BC Gen. Jnl.-Post Line, posting engine). iDempiere's accounting facts are a regenerable post-completion projection (`Doc.createFacts`, re-posting drops and recreates) (Doc Javadoc).

In ObjectFlow this is a Status variant, `document status`, that predefines the lifecycle vocabulary and hook slots (before/after prepare, complete, void, close, reverse), a result-status contract per action, the expected-status assertion, a re-read-under-lock of the Status before an action, and a `period control` Precondition family keyed by document type, date, organisation and action. A `GRAPH_OWNER_CMD` kind that produces append-only entries, with ManMap mappings marked `APPEND_ONLY` so the checker rejects `UPDATE/DELETE` on them, captures the ledger pattern.

In MoWare today. Each aggregate re-implements its lifecycle as Service Methods with hand-chosen Status elements, and cross-aggregate effects of completion (create shipment, post entries) are coded in the same methods or in Commands; nothing distinguishes a mutable master-data Entity from an immutable posted entry, and nothing prevents a `delete with` on one (MM § "Löschen mit `delete with`"; CONV `mw-domaenenmodell_v1.md`).

B3. Domain events with guaranteed post-commit dispatch and a publication registry

Every surveyed platform insists that notifications fire only after commit: RAP business events are raised with `RAISE ENTITY EVENT` in the late save "after the point of no return" and consumed asynchronously (SAP Learning events); Business Central's external business events are "sent to external systems only after the

current transaction is committed" and its isolated events run each subscriber in its own transaction so a failing subscriber cannot roll back the publisher (BC business events, BC isolated events); OFBiz's `global-commit-post-run` SECA with `statusId != oldStatusId` is the idiom for "react to a state change after commit" (order secas.xml); metasfresh's `afterCommit = true` warns that work in such a hook needs its own transaction and that failures are only logged (ModelChange.java); Odoo automation rules trigger on `on_state_set`, `on_create_or_write` and date-based conditions with pre- and post-domain filters (base_automation.py). Axon shows the two dispatch modes a DSL must name: a subscribing processor runs in the publisher's thread and transaction, a streaming processor runs in its own thread with a persisted tracking token, retries from 1 to 60 seconds and dead-letter queues, and Axon's state-stored aggregate proves that events can be published from a state-persisted entity without event sourcing (Axon state-stored aggregates, event processors). Spring Modulith's event publication registry is the minimal reliable in-process form: it "writes entries for each [listener] ... as part of the original business transaction", marks them completed after the listener succeeds, tracks attempts and statuses (PUBLISHED, PROCESSING, COMPLETED, FAILED, RESUBMITTED), republishes outstanding entries on restart, and its own docs concede the native externalization "lacks critical features developers might expect from actual outbox pattern implementations" (Spring Modulith events). The transactional outbox pattern proper stores the outgoing message in the business transaction and lets a relay publish it, guaranteeing "sent if and only if the database transaction commits" at the price of duplicates, so consumers must be idempotent; its stated drawback, that "the developer might forget to publish", disappears when the DSL writes the row (microservices.io outbox). The consistent advice is to adopt events without event sourcing: Dudycz lists CRUD systems, data-bag modules and inexperienced teams as poor fits and stresses "it's not an all-or-nothing decision" (Dudycz); Vernon's rules "model true invariants in consistency boundaries" and "use eventual consistency outside the boundary" are the DDD argument for events rather than wider Sessions (Vernon Part II).

In ObjectFlow this is an `event RechnungFreigegeben(rechnungKey, betrag)` root, an `emit` statement in Service Methods, and `on event RechnungFreigegeben (in session | after commit)` handler declarations with an ordering and error policy (log-and-continue, propagate, dead-letter). The Session writes the registry or outbox rows in the same transaction as the check-in, so "emit on commit" is a guaranteed semantic; a `consumer/producer pair` or a relay drains the outbox for external consumers. Test support follows Modulith's `AssertablePublishedEvents`: a `run command` can assert that an event with given properties was published.

In MoWare today. Domain events do not exist; the workbench documentation says the stack "übernimmt aber nicht sämtliche DDD-Konzepte" (WB § "Zentrale Konzepte der MoWare-Werkbank"). The nearest mechanisms are `session queue next command`, which schedules one UI Command after commit, is ignored in tests and jobs and "ersetzt jedoch keine allgemeine Job- oder Workflow-Engine" (OF § "Command nach dem Commit einplanen"), and `TO_SESSION_OPS` Service Methods that join the owner transaction. Cross-aggregate reactions are coded in the committing Command, and a rule spanning two aggregates collides with the no-cycle convention and is resolved by the caller passing facts (PR2 P-050). A search refreshing after an edit needs a hand-written `any cmd term` handler (PR2 P-044).

B4. Persisted process instances, sagas and user tasks

Camunda 8 persists process state as a BPMN model: a job is created when execution reaches a task, pulled by job workers with at-least-once delivery and explicit retries, exhausted retries raise an incident until resolved, and user tasks carry assignee, candidate groups, due date, priority, form binding and listeners while "the process instance stops at this point and waits" (Camunda job workers, user tasks). Temporal makes arbitrary deterministic code durable by replaying an event history, at the cost of determinism rules, patch-based versioning and hard history limits of 51,200 events or 50 MB that force continue-as-new; its message model splits Signal (async write), Query (read, no history) and Update (synchronous, validated before acceptance) (Temporal workflow definition, limits, message passing). Eventuate's saga DSL is small enough to transcribe:
`step().withCompensation(reject).step().invokeParticipant(reserveCredit).step().invokeParticipant(approve)`,
forward steps sequential, compensations in reverse (Eventuate sagas); Microsoft's saga guidance adds the pivot/retriable classification and countermeasures such as the semantic lock, which is literally a status value marked pending (Microsoft saga). Axon sagas are routed by an association property with

`@StartSaga`/`@EndSaga` (Axon sagas). OutSystems launches a process declaratively on an entity's Create action and closes a human activity "on a `Create<Entity>`/`Update<Entity>` event with attribute conditions" (OutSystems Process, Human Activity); Mendix workflows carry a `$WorkflowContext` entity through user tasks, decisions and timers (Mendix workflows). Axelor and Jmix delegate to BPMN engines (Flowable), iDempiere has a WfMC-style data-defined engine, and Business Central's engine is assembled from runtime tables, a cautionary example of what happens when the process concept is not in the language (BC workflow walkthrough). The code-first versus BPMN comparison rests on vendor and consultant pages, but the structural point stands: MPS already gives ObjectFlow a projectional, graphically renderable model, so the shared-visual-model argument for BPMN can be had without a second XML artefact.

In ObjectFlow a `process RechnungsFreigabe keyed by rechnungKey` root with ordered steps (local Service call, user task offered to a role with due date, wait for an event or deadline, call to another aggregate with a compensation), a persisted step position and typed process data, a retry policy per step, and start policies in Temporal's vocabulary (reject duplicate, use existing) would replace the one-hop mechanisms. A user task maps onto an existing Command offered to a role whose `FINAL OK_CONCLUSION` resumes the process; a wait step completes when an Entity reaches a declared Status (B1) or an event (B3) arrives. Persisting the position rather than replaying code avoids Temporal's determinism ceremony, but running instances still outlive the code that created them, so step insertion needs a declared migration rule, as Camunda's versioned deployments do.

In MoWare today. Multi-step flows are expressed as successor Commands, which share the Session, defer the predecessor's commit until the successor succeeds and explicitly do not support chains ("Eine weitere Successor-Kette am Ziel-Command wird nicht unterstützt"), or as `session queue next command`, which is UI-only (OF § "Successor-Commands"; § "Command nach dem Commit einplanen"). Anything spanning days, users or restarts is a Status on the Entity plus a `consumer/producer pair` whose producer query re-discovers the work (DUX § "Batchjob mit `BatchJob Module`").

B5. Time as an event source: deadlines, timers and schedules

Three constructs recur. Deadlines scoped to an instance: Axon's `deadlineManager.schedule(Duration, "name", payload)` with a `@DeadlineHandler` in the same aggregate or saga, cancellable by id or by scope, and deliberately not stored as an event (Axon deadline managers). Persisted timers inside a process: Temporal timers "are persisted", cost nothing while waiting, run from one second to several years and are treated as a minimum duration (Temporal timers); XState's delayed `after` transitions and Spring Statemachine's `timerOnce` are the state-machine form (XState transitions, Spring Statemachine); Camunda user tasks carry `dueDate`/`followUpDate` (Camunda user tasks); Odoo automation offers `on_time`, `on_time_created`, `on_time_updated` triggers checked by a cron every four hours by default (Studio automation rules). Schedules as first-class objects: Temporal's Schedule has its own identity, overlap policies (Skip, BufferOne, BufferAll, CancelOther, TerminateOther, AllowAll), a catch-up window, pause and backfill (Temporal schedules); OutSystems timers declare schedule, a 20-minute default timeout, retries, `Wake<Timer>` and never run twice concurrently (OutSystems Timer, ODC timers); Business Central's Job Queue adds recurrence, logging, an error-handler codeunit in a new session and a maximum number of attempts (BC job queue). Axon notes that a db-scheduler-style single table is the lightweight persistence option and that Quartz "has seen little recent activity" (Axon event schedulers).

For ObjectFlow this is a `deadline` declaration on an Entity Status or process step (`in AWAITING_PAYMENT after 14 days -> EXPIRED unless cancelled by zahlungEingegangen`), a `wait until` step in processes, a due date on user tasks, and a `schedule` object replacing the bare `CRON`/`DELAY` options with overlap policy, catch-up window and timeout. All of them need one persisted scheduler table in the runtime.

In MoWare today. Time exists only as `CRON` and `DELAY` on a `consumer/producer pair` in the JVM default timezone; there is no per-element timeout, no overlap policy beyond "producer runs only when no consumer is working", and any deadline is a producer query over date columns that rediscovers overdue work each run (DUX § "Kapitellandkarte: Batchoptionen"; § "Batchjob mit `BatchJob Module`").

B6. Durable work queue with retry policy, incidents and idempotency

metasfresh's workpackage queue persists elements and parameters in `C_Queue_WorkPackage`/`C_Queue_Element`, binds a package to the enclosing transaction so it becomes ready only after commit (`bindToThreadInheritedTrx`, `markReadyForProcessingAfterTrxCommit`), carries client, organisation and user context from enqueue time, and gives processors `isAllowRetryOnError()` and a skip request (IWorkPackageBuilder.java, IWorkpackageProcessor.java). OFBiz persists async service calls in `JobSandbox` with `max-retry` and iCalendar recurrence (OFBiz Service Engine Guide); Odoo's cron protocol processes one batch, commits per batch via `_commit_progress`, skips after three consecutive errors and deactivates the job after five failures over seven days (Odoo actions). Temporal's retry policy fields are the canonical set: initial interval 1 s, backoff 2.0, maximum interval, maximum attempts, non-retryable error types, overall schedule-to-close timeout, with the rule that activities retry by default and workflows do not (Temporal retry policies). Camunda names the terminal state: exhausted retries raise an incident, resolution re-evaluates (Camunda job workers). Idempotency is the consequence of at-least-once everywhere; Stripe's contract (client key, first result stored and replayed on retry including 500s, parameter mismatch rejected, keys pruned after 24 hours) is the cleanest specification (Stripe idempotent requests), and Temporal's workflow-id reuse and conflict policies specify idempotent process start (Temporal workflow id).

In ObjectFlow the `consumer/producer pair` would gain a persisted inbox option with element status, attempt count and incident state; a reusable `retry policy` declaration with Temporal's fields attachable to consumers, event handlers and process steps; an `incident` reaction in the exception strategy next to `READD_TO_INBOX`; and `idempotent by <key expression>` on Commands and Service Methods generating a processed-key table. A third problem axis, transient versus permanent technical failure, extends the existing Precondition/Guard split and feeds the retry decision.

In MoWare today. "Die Inbox ist eine flüchtige In-Memory-Queue und keine persistente Jobwarteschlange": it is cleared before every producer run, lost on restart, an element is removed before processing and re-added only with `READD_TO_INBOX`, there is no retry otherwise and no per-element timeout; consumers must be idempotent by re-checking in a Precondition that the work is still needed (DUX § "Batchjob mit `BatchJob Module`"; § "Exception-Strategien und Wiederanlauf").

B7. Draft: a persisted, resumable, lockable unit of work (optional)

RAP's draft is the one mechanism with no MoWare analogue: `with draft;` gives every entity a shadow draft table, implicit actions Edit, Activate, Discard, Resume and a Prepare that runs on-save determinations and validations, an exclusive lock on the active instance "not bound to the ABAP session" with a 15-minute default lifetime followed by an optimistic phase guarded by a total ETag, and automatic removal after 28 days of inactivity; it exists because "stateless RESTful protocols such as OData have no fixed session" (SAP Learning draft, SAP cheat sheet 36). The cost is a shadow table per entity, `%is_draft` in every key and sharp edges with legacy keys reported in the community. Causeway's command log records every top-level action and `BackgroundService` persists a command for later execution (Causeway command log).

For ObjectFlow a `GRAPH_OWNER_CMD` option `DRAFTABLE` could serialize the Session graph (entities, dirty state, pending session operations) to a draft table at user-cancel-with-save and resume it later with the lock rules above, reusing the existing serdes. It is expensive and should remain opt-in.

In MoWare today. The Session is in-memory and dies with the Command; `revert` is in-memory only, `acquire locks` are released on every termination, and a long edit interrupted by a session loss restarts from the database state (OF § "Revert beim Abbruch"; § "Pessimistische Sperren mit `acquire locks`").

Theme C. Persistence: what ManMap could declare

C1. Declared composition with generated graph save, orphan delete and tree versioning

RAP distinguishes a composition from an association by lifecycle: "a child entity ... only exists together with its parent", deleting a managed parent deletes composition children, and the root is lock master, authorization master and `total etag` owner for the whole tree while children declare `lock dependent by _Parent` (SAP Learning composite BOs, SAP cheat sheet 36). Jmix's `@Composition` means child and owner are edited and saved together in one transaction, with `@OnDelete(DENY|CASCADE|UNLINK)` and `@OnDeleteInverse` as declared delete policies (Jmix entities). Blaze-Persistence requires updatable views to declare cascade and orphan removal explicitly and to match the entity's settings or fail (Blaze manual); Hibernate's `StatelessSession` operations "do not cascade" (Hibernate Introduction). The critique of default cascading is consistent: cascaded REMOVE on many-to-many "might end-up deleting more than we're expecting" and is often discovered after the fact (Vlad Mihalcea cascade types); Axelor's one-to-many defaults to `orphanRemoval=true`, the opposite stance (Axelor models). Business Central codes child deletion explicitly in the header's `OnDelete` (BC data triggers).

For ManMap the fitting form is an explicit, non-transitive `composition` option on a `ListMapping` (and on a `ReferenceMapping` for owned single children) from which the generator emits a `save graph with <Mapping> (root)` that saves the parent, propagates the key or back-reference, saves each child, and deletes children whose keys were loaded but are no longer in the list; a `delete graph with` in children-first order; a `DENY`/`UNLINK` policy for references from outside the aggregate; and an `OPTIMISTIC_LOCK` scope over the whole tree (RAP's total ETag) so a child edit bumps the root's TCN. Plain `ReferenceMapping`s stay cross-aggregate, read-only references as the conventions already demand.

In MoWare today. `save with` saves exactly one mapping's columns; the developer writes "zuerst der Parent gespeichert, dann der Parent-Schlüssel beziehungsweise die Rückreferenz an die Kinder gesetzt und schließlich jedes Kind über sein eigenes `EntityMapping` gespeichert", deletes children explicitly before the parent, and detects removed children by remembering the loaded list and diffing it in `FINAL OK_CONCLUSION`, a workaround documented as not a moai topic (MM § "Speichern von Objektgraphen"; § "Löschen mit `delete with`"; PR2 P-041). Every aggregate repeats this in its `<Aggregat>Repo.checkin`.

C2. Read models whose SELECT and joins derive from a declared shape

The dominant idea across the persistence ecosystem is that the *shape* drives the query. Blaze-Persistence entity views are interfaces of getters over an entity (`@EntityView(Cat.class) interface CatNameView { @IdMapping Long getId(); @Mapping("name") String getCatName(); }`), with nested subviews, collection mappings, `@Limit(limit="5", order={"age DESC"})` per collection, attribute filters and keyset pagination, and the engine derives the SELECT and joins; its GraphQL integration skips joins for unrequested fields (Blaze manual). Spring Data closed interface projections let the engine "optimize the query execution, because we know about all the attributes" (Spring Data projections). jOOQ's `MULTISET` collects a child subquery into a nested collection in one statement, native on Oracle and emulated via JSON on MariaDB/MySQL, and implicit path joins (`BOOK.author().FIRST_NAME`) derive the JOIN from the FK graph with "no performance penalty" (jOOQ MULTISET, jOOQ implicit join). The ERPs have the declarative read model as data: OFBiz view-entities with member entities, `view-link` (outer join via `rel-optional`), aliases with `group-by` and `function="sum|count|..."` (OFBiz Entity Engine Guide); Business Central Query objects with `DataItemLink`, `SqlJoinType` and `Method = Sum|Count|...` that implicitly GROUP BY the other columns (BC Query object, aggregation); iDempiere Info Windows with a FROM clause, criteria columns carrying operator and SQL function, identifier columns, paging and role-based access-filter injection (iDempiere Info Window). OutSystems aggregates fetch only the attributes actually used and survive renames because attributes reference the model (OutSystems Aggregate); OpenXava's `@Tab` is a named list read model with columns including paths, a base condition and default order (OpenXava tab). Result windows belong to the same idea: Spring Data separates Page (count query), Slice (size plus one probe) and keyset Window, which "rewrites the criteria clause to include all sort fields and the primary key" and needs non-null sort keys (Spring scrolling, query methods); jOOQ's SEEK emulates row-value predicates per dialect and cannot be combined with OFFSET (jOOQ SEEK).

For ManMap this is a `view <Name> on <EntityMapping>` concept inside a Persistence Description whose members are dotted paths over mapped fields and references (`kunde.name`), nested list views with an optional per-parent limit, and aggregate members with `group by`; a `READONLY` repository method `load view RechnungInfo where ... window (page | slice | keyset)` has its SELECT, joins (INNER for non-nullable references, LEFT otherwise), nested collection fetch (MULTISET on Oracle, JSON aggregation or a second IN-list query on MariaDB/MySQL) and count or seek predicate generated. MPS can check projection, join paths and bind arity in the editor, which is the structural gain jOOQ offers over strings. The target type is a DTO, keeping the read model outside the identity map as today.

In MoWare today. A `where` filter cannot aggregate, project or join beyond declared `refJoin`/`listJoin`, so every search and list view is a `<Thema>LeseRepo` with a hand-written `SQL query`, dynamic `if` fragments per criterion, and a `nokeystore/read-only map` or `row mapper` into a DTO whose property list must be kept in sync by hand (MM § "Custom SQL mit `SQL`"; § "Row-Mapper und No-Key-Mapper"; CONV `mw-anwendungsaufbau_v1.md` § "`domain`, `read` und `unit`"). SQL text is "Wort für Wort ein eigener Knoten" in the model and binding a Status variable was unclear enough that a search was rebuilt as a mapped `where` (PR2 P-033). Paging is `limit` only, with no count or keyset support (MM § "Gemappte Abfragen mit `QueryFromMap`").

C3. Declared aggregate and derived properties

Business Central FlowFields "aren't physical fields ... They're a description of a calculation": `Sum("Customer Entries".Amount where(CustNo=field(CustNo)))`, `Count`, `Exist`, `Min`, `Max`, `Lookup`, computed by SQL on demand, parameterised by user-settable FlowFilters, and accelerated by SIFT indexed views or columnstore indexes; the documented pitfalls are that hidden FlowFields were computed anyway until v26, that FlowFields read in code "are always zero" unless `CalcFields` is called, and that SIFT maintenance has write overhead and locking issues (BC FlowFields, FlowFilters, SIFT and SQL Server, SIFT to NCCI). Odoo computed fields declare `@api.depends('value','tax')`, default to not stored, and with `store=True` become searchable and groupable; a single-author blog documents recomputation storms from over-broad dependencies (Odoo ORM, dev.to). OpenXava separates three flavours: `@Calculation` expression over same-entity properties recalculated live, a getter-only calculated property with `@Depends`, and `@Formula` as an SQL fragment computed by the database and filterable in lists (OpenXava model); Axelor has formula (SQL) versus calculated (Java) fields (Axelor models); iDempiere has `Column SQL` virtual columns (ADempiere AD overview); Hibernate's `@Subselect`/`@Immutable` maps a read-only entity to a SQL select (Hibernate @Subselect).

ObjectFlow already has virtual Business Properties; the addition is an `aggregate` kind whose body is a declared aggregate over another EntityMapping (`sum(Zahlung.betrag) where Zahlung.rechnung == this`), computed by a generated SQL statement only when a repository method explicitly requests it (`load ... with aggregates(offenerBetrag)`), never implicitly, and optionally parameterised by caller filters, with an `INDEX` hint option. A `depends on` list for in-memory derived properties lets the generator invalidate or recompute and lets the checker reject self-dependency. The BC lesson, that implicit calculation is what bites, matches ManMap's explicit style: the DSL should make the calculation visible at the call site.

In MoWare today. A virtual property's `get` runs on every read and "können keine Repositories oder Services aufrufen", so any aggregate over child rows is either loaded as the full list and summed in memory or fetched by a hand-written `SQL query` into a DTO, and totals shown in lists are columns of a `<Thema>LeseRepo` read model (OF § "Virtuelle Business Properties"; MM § "Lesemodell mit Custom SQL verwenden").

C4. Named load graphs per use case

Jmix fetch plans declare "what object graph should be eagerly loaded ... in a particular operation", are named and reusable (`_local`, `_base`, custom plans in `fetch-plans.xml`, `extends`), and leave omitted attributes empty so access throws "Cannot get unfetched attribute" (Jmix fetching). Hibernate's guide calls JPA entity graphs "untypesafe and unnecessarily verbose" but values the idea, and offers fetch profiles that selectively request subselect fetching and batch fetching by IN-lists as the only alternative to joins that needs no lazy proxies (Hibernate Introduction). The critiques of lazy loading are unambiguous: N+1 is

missed by slow-query logs because each query is fast, and Open Session in View holds the connection through rendering, so "a DTO projection forces the application developer to fetch just the required data set" (Vlad Mihalcea N+1, OSIV). SOQL expresses the same per query with parent dot-paths and child subqueries (Trailhead SOQL).

ManMap's "unloaded reference throws, unloaded list is empty" is already Jmix's partial-entity contract, so a `load graph <Name> on <EntityMapping> { kunde; positionen { artikel } }` declaration attached to `CHECKOUT/READONLY` methods, with a strategy per edge (join, IN-list batch, separate query), composes with the no-lazy-loading stance; the same graph name can be referenced by DataUX bindings and by an API contract. A checker can warn when a method's body touches a reference not in its declared graph.

In MoWare today. Each `checkout(id)` method loads the root with `get`, then assigns references and lists by separate queries or declares `refJoin/listJoin` per query, and "Eine leere Liste beweist daher nicht, dass in der Datenbank keine Kindzeilen vorhanden sind"; when a reference is not loaded the code reads `#Key` and calls `# Repo.get(...)` by hand (MM § "Explizites Laden"; OF § "Beziehungen und Objektgraphen"). The graph loaded is implicit in the method body and cannot be reused by name.

C5. Implicit row restrictions: soft delete, tenancy, named filters and row-level security

One mechanism covers four needs. Hibernate 6.4 made soft delete declarative (`@SoftDelete` with strategy ACTIVE/DELETED, indicator column and converter; delete becomes an UPDATE and entity queries hide the rows, native SQL does not), made discriminator multi-tenancy a `@TenantId` column with automatic filtering ("native SQL queries are not automatically filtered by tenant id"), and offers named, parameterized `@FilterDef/@Filter` restrictions enabled per session and deliberately not applied to `find()` or `@ManyToOne` (Hibernate User Guide §3.12, Hibernate Introduction). Jmix documents the soft-delete sharp edges: unique constraints need DB-specific tricks and to-one references may dangle to deleted rows (Jmix soft deletion). Row-level security is a declared predicate per role in Jmix (`@JpqlRowLevelPolicy(where="c.active = TRUE")` with `{E}` alias and session parameters, root entity only), Odoo (`ir.rule` domains, global rules AND, group rules OR), iDempiere (`MRole.addAccessSQL` appending client/org and access clauses to every Info Window and report query), Mendix (XPath constraints on access rules applied at retrieve and hidden from the client) and Business Central (security filters on permission sets plus a per-record `SecurityFiltering` mode Filtered/Validated/Ignored, where Validated is slow and Ignored silently bypasses) (Jmix row-level roles, Odoo security, MRole.java, Mendix access rules, BC security filters). Causeway's SecMan derives its permission tree from the metamodel but admits its tenancy is "only skin deep" (Causeway SecMan).

For ManMap a `restriction <Name>(params) on <EntityMapping> where <mapped predicate>` concept, with `SOFT_DELETE(column, strategy)` and `TENANT(column)` as predefined restrictions and role-bound restrictions declared in the `Roles and Permissions` root, would be ANDed into every generated `get/where/reload` and set the column on INSERT, with a per-method mode (`filtered`, `unrestricted`) and the documented rule that custom `SQL` blocks are the developer's responsibility. Soft delete turns `DELETE` methods into UPDATEs and adds a `deleted also` query modifier for administration.

In MoWare today. Visibility is a Scope function returning accessible objects of a type, bound into Page metadata via `setScope`, evaluated in the Command and not in the Repository, so every Repository query that must respect it repeats the predicate; `CAN_OPEN_RO/CAN_OPEN_RW` gate Commands, not rows (OF § "Rollen, Scopes und Identities"; § "Scopes"). There is no soft delete and no tenancy column convention; an `IsActive`-style flag is an ordinary Status property that every `where` must remember to test.

C6. Historisation and business change tracking

The database route is cheapest where available: MariaDB `WITH SYSTEM VERSIONING` creates a history row on every UPDATE of a versioned column, answers `FOR SYSTEM_TIME AS OF`, `BETWEEN` and `ALL`, partitions history with `PARTITION BY SYSTEM_TIME`, and adds `row_end` to the primary key with replication and dump caveats (MariaDB system-versioned tables); Oracle offers Flashback Query `AS OF TIMESTAMP`, Version Query with `VERSIONS_*` pseudocolumns and Flashback Time Travel archives that forbid adding foreign-key constraints (ORA-55610) (Oracle Flashback); MySQL has neither. The portable application route is Envers: `@Audited` writes a `_AUD` row per change with `REV` and `REVTYPE`, the `ValidityAuditStrategy` adds `REVEND` so point-in-time

reads become a range condition instead of a correlated `MAX(REV)` subquery, and `AuditReader.find(Class, id, Date)` returns the state at a time (Vlad Mihalcea Envers, AuditReader). Fowler advises against bitemporal modelling unless retroactive change is a real requirement (Fowler bitemporal). Business change tracking is a different need: Axelor's `<track><field name="confirmDate" on="UPDATE" if="status == 'confirmed'"/> <message if="status == 'confirmed'" on="UPDATE">Order confirmed.</message></track>` produces conditional, human-readable messages threaded on the record, asynchronously after the transaction (Axelor messaging); Odoo's `tracking=True` per field writes `mail.tracking.value` rows (Odoo mixins); iDempiere's "Maintain Change Log" and Business Central's Change Log write old/new values per column, the latter with the warning not to log ledger tables because of volume (PO.java, BC change log); Causeway pairs a command log (intention) with a property-level audit trail (effect) correlated by interaction id (Causeway audit trail, command log); Jmix's Entity Log is configured per entity and attribute but blind to native SQL (Jmix entity log).

For ManMap an `AUDITED` table option that generates the validity-layout `_AUD` table and REVINFO (reusing the `ADDITIONAL_TABLE_NAME` machinery), the extra INSERT in `CHECKIN/DELETE`, and an `as of <timestamp>` modifier on `READONLY` methods is the portable choice across Oracle, MySQL and MariaDB; a `SYSTEM_VERSIONED` option can emit the MariaDB clause (and Flashback Archive on Oracle, with the FK caveat) where the database is known. For ObjectFlow a `track` block on an Entity with per-property `on create|update if <cond>` and templated messages, written at check-in, is the business-facing counterpart. Correlating both with the committing Command's name and the Session's user gives Causeway's intention-plus-effect pairing for free, because the Session already knows both.

In MoWare today. ManMap offers `CREATEDAT/CREATEDBY/MODIFIEDAT/MODIFIEDBY` and `OPTIMISTIC_LOCK` (whose required extra `TCN` column was undocumented, PR2 P-013) and nothing else: no history table, no as-of query, no change log (MM § "Optimistic Locking und Audit"). Applications needing "who changed what" write their own log Entity and insert into it from every transition Service Method.

C7. Model-driven schema evolution and checked deprecation

Both mainstream tools version the schema as an ordered list of immutable change units in a history table: Liquibase changesets (id, author, path) with preconditions, rollback and `DATABASECHANGELOG`, and a `diff-changelog` that compares two schemas, or a Hibernate model against a database via the `liquibase-hibernate` extension, and whose own wiki shows that a renamed class yields a new table rather than a rename (Liquibase changelogs, diff-changelog, liquibase-hibernate wiki); Flyway versioned, repeatable and undo migrations with checksummed, immutable scripts (Flyway migrations; naming conventions from secondary sources). Hibernate's `hbm2ddl update` "is to be avoided" in production (Vlad Mihalcea hbm2ddl). Business Central shows evolution without a migration DSL: `ObsoleteState` Pending yields compiler warnings and Removed yields errors on references, upgrade codeunits with upgrade tags handle data, and schema changes are additive-only online ("Don't delete primary keys ... Don't add more unique keys") (BC ObsoleteState, upgrading extensions, table keys); metasfresh records every Application Dictionary change as a timestamped SQL migration script (log_migration_script.md).

The structural advantage of a DSL is that the model history knows *intent*: a rename, a type widening or a split of an embedded Value Object are visible in a model diff but not in a database diff. The realistic ManMap design is a generator that emits a reviewed Liquibase XML or Flyway SQL changeset per dialect from the diff between two model versions (or a model and a database snapshot), tracked by the standard history table and never applied automatically, plus a `DEPRECATED` option on mappings and properties with Pending/Removed states that the checker turns into warnings and errors as BC does.

In MoWare today. "ManMap ist kein allgemeines Werkzeug zur Migration bestehender Datenbankschemata"; the developer creates the schema from the generated description in MPS, schema hints are explicitly "keine Schema-Migration", the generated description uses Oracle syntax (PR2 P-023), `INDEX` silently generated a unique index (PR2 P-022), and every later change is a hand-written ALTER (MM § "Datenbankportabilität und Schema"; WB § "Von der Modellierung zur Ausführung").

C8. Set-based writes and dialect-emulated upsert

JDBC batching "only helps in cases where a program executes many inserts, updates, or deletes" against one table, and set-based HQL `update/delete` are "even better" (Hibernate Introduction); Spring Data's `UpdateSpecification/DeleteSpecification` reuse query predicates for bulk statements (Spring Data Specifications); jOOQ renders one upsert declaration as `ON DUPLICATE KEY UPDATE` on MySQL/MariaDB and as `MERGE INTO ... USING` on Oracle (jOOQ upsert); MyBatis Dynamic SQL refuses to render a statement whose optional WHERE conditions all dropped, because it "could end up impacting all rows" (MyBatis Dynamic SQL).

For ManMap this is a mapped `update <EntityMapping> set <field> = <expr> where <mapped predicate>` and `delete <EntityMapping> where ...` repository method kind with the empty-WHERE guard, and an explicit `UPSERT` save variant per EntityMapping that the generator emulates per dialect; because upsert bypasses the TCN check it must be a separate method type, not an option on `CHECKIN`.

In MoWare today. Set-based changes are a `STATEMENT` SQL block with hand-written dialect-bound text; `BATCH` exists but is unavailable on MySQL/MariaDB and fails with auto-IDs there (MM § "`SQL query` und `SQL statement`"; § "Save-Optionen"; PR2 P-040).

Theme D. Contracts, problems and modules

D1. Declared problems on Service Methods and a derived API contract

Rust's position is that `Result` is the default "when you're defining a function that might fail" and `panic!` is for contract violations that indicate a caller bug, with `?` propagating and converting errors (Rust book ch09-02, ch09-03); Kotlin dropped checked exceptions and recommends null-returning variants for expected absence (Kotlin exceptions); GraphQL returns `data` plus an `errors` list with `message`, `path` and `extensions`, propagating a non-null field's failure to the nearest nullable parent (GraphQL spec); OpenAPI declares per-status-code response schemas with `4XX/5XX` ranges and a `default` (OAS latest). Error accumulation is what UIs need: Iron's `refineValidation` accumulates, Mendix records all failing rule messages, Business Central's `[ErrorBehavior(ErrorBehavior::Collect)]` collects instead of stopping (Iron, Mendix validation rules, BC collecting errors). Mendix's `Validation Feedback(object, member, message)` and Salesforce's field-anchored validation message show that a problem should name the offending property (Mendix validation feedback, Trailhead validation rules). Mendix derives an OData v4 service with an OpenAPI 3.0 document and optional GraphQL schema from entities and microflows; Jmix's generic REST reads the same security as the UI; Causeway implements Restful Objects over the metamodel (Mendix published OData, Jmix REST, Causeway RO viewer).

The ObjectFlow part is a `problems { NichtGenugBestand, KundeGesperrt }` clause on Service Methods so the expected failures become part of the signature and generated code returns a sealed outcome; a `problem(property, message)` form of Precondition anchoring the message to a Business Property; and a collect-all mode for Preconditions in direct Service calls. From these plus A3 (nullability), A4 (sealed DTO alternatives as `oneOf`/union) and a page or cursor DTO convention, an OpenAPI 3.1 document for every `API_METHOD` is a generator output with the existing JSON serdes as implementation; additive evolution with `deprecated` flags is the cheaper versioning path, as GraphQL's versionless practice suggests (GraphQL schema design). API derivation itself is in scope only this far; the UI and transport layers are not.

In MoWare today. `API_METHOD` "markiert für API-Integration" and changes nothing else; there is no contract artefact (OF § "Service-Methoden"). A violated Precondition in a direct Service call throws `OFXAbortedException` and leaves the problems on the Session so later calls in the same Session fail too (PR2 P-030); a Precondition message is a `StringFormatString` with no property anchor, and `validation` collects violations only inside a Command flow (OF § "Preconditions, Validation, Guards und Exceptions").

D2. Module boundaries as checked constraints and a generated module canvas

Spring Modulith treats each sub-package as a module with a provided interface (API beans and published events), internals that "must not be referred to from other modules" and declared allowed dependencies; `ApplicationModules.of(App.class).verify()` fails on a cyclic dependency graph or an internal reference, and its `Documenter` emits C4 diagrams and a per-module canvas listing aggregates, published and consumed events and configuration (Modulith fundamentals, verification, documentation). Causeway's SecMan derives a namespace/type/member permission tree from the metamodel with "most specific wins" (Causeway SecMan). OutSystems refuses to make an action public if its parameters use non-exposed entities (OutSystems Server Action).

In a projectional DSL these become type-system errors rather than tests: the conventions' import table (`base`, `rollen`, `sharedvalues`, `gate.*`, `<bereich>.domain`, `read`, `unit`, `app`, `tests`), the no-cycle rule between bounded contexts and the DTO placement rules can be declared once per model as a `layer` and checked at edit time; a generated canvas (Entities, Commands, Services, events published and consumed per Bereich) follows directly from the model and would supersede the hand-maintained overview tables in the conventions.

In MoWare today. Layering is "strict import table; no higher-level imports; no cycles between bounded contexts" by convention and review only (CONV `mw-anwendungsaufbau_v1.md` § "Solutions und Modelle", § "Abhängigkeiten"); a violation surfaces as a design discussion (PR2 P-050), not as a checker error.

Three runtime capabilities carry most of the ideas

Read together, the themes reduce to three investments in the runtime and generator, each of which unlocks several ideas at once. First, making the Session a *phased* unit of work with named hook points (determinations, validations, save, post-commit) turns A2, B3, C6 and the business-tracking half of C6 into generated code at check-in, because the Session already owns the transaction and already knows the user and the Command; RAP's strict mode and Salesforce's fixed pipeline both show that naming the phases is what makes rules composable and checkable. Second, a *persisted scheduler and work queue* (one or two tables in the style of db-scheduler and metasfresh's workpackages) is the shared substrate for B4, B5, B6 and the registry drain in B3; without it every "later" in a MoWare application is a producer query that rediscovers work. Third, a *graph-aware ManMap generator* that reads the declared FK graph is what makes C1, C2, C3, C4 and C8 mechanical, and MPS's reference-based editing already gives the rename-safety that OutSystems and jOOQ advertise.

Equally important is what the evidence says *not* to copy. Lazy loading (N+1, OSIV), implicit cascade (Vlad Mihalcea's critique, Axelor's `orphanRemoval` default), implicit aggregate calculation (BC's hidden FlowFields), silently skippable rules (BC's `RunTrigger`), rule engines with agendas (Drools, Easy Rules) and event sourcing as the source of truth (Dudycz, Fares) are all documented pain points; the MoWare principles of explicit loading, explicit save and check-before-change are vindicated. The ideas above are therefore framed as *declared, visible, non-transitive* variants: a composition that generates the parent-then-children statements you would have written, a view whose SELECT you can read, an aggregate computed only where requested, rules that always run at check-in, and events as notifications while state stays the truth.

Suspected recurring hand-written work

The baseline note's section 5 lists the structures every application re-implements per aggregate, per search or per test model, and section 6 lists the language limits users actually hit in the two problem logs; the conventions' naming recommendations (`<Aggregat>Repo`, `<Aggregat>Service`, `<Bereich>Persistenz`,

`<Bereich>Lesen`, `<Thema>LeseRepo`, `<Bereich>Tests`) are themselves evidence that these artefacts recur identically. The table maps each to the idea or ideas that would absorb it. Where an item is only partially absorbed, the remainder is noted.

Hand-written pattern (what is written each time)	**Where documented**	**Absorbed by**
Search filter DTO with `results: list<Info-DTO>`, constructor resetting Status properties to `null`	WB § "Gesamtbeispiel", § "Rechnungen suchen"; PR2 P-034	C2 (view with filter parameters and result window), A3 (nullable Status needs no constructor reset)
Search Command with filter page and result page, `page <same page>` as refresh	WB § "Rechnungen suchen"; OF § "Page Conclusions"	C2 (generated view query); refresh after edit via B3 (event handler re-runs the view) instead of `any cmd term` (PR2 P-044)
`<Aggregat>Repo` with `checkout(id)` loading root plus lists/references and `checkin(entity)`	WB § "Persistenz mit ManMap"; CONV `mw-anwendungsaufbau_v1.md`	C1 (`save graph with`), C4 (named load graph on `CHECKOUT`)
Parent-then-children save ordering, key or back-reference propagation	MM § "Speichern von Objektgraphen"	C1
Children-before-parent delete	MM § "Löschen mit `delete with`"	C1 (`delete graph with`), with `DENY/UNLINK` for outside references
Removed-children detection by remembering loaded list and diffing in `FINAL OK_CONCLUSION`	PR2 P-041	C1 (orphan delete by key diff)
`<Thema>LeseRepo` with `SQL query` and `nokeystore/read-only map` into DTO; separate `<Bereich>Lesen` Persistence Description	WB § "Persistenz mit ManMap"; MM § "Lesemodell mit Custom SQL verwenden"	C2; aggregate columns by C3
Dynamic SQL via `if` per criterion	MM § "Custom SQL mit `SQL`"	C2 (optional predicates in views) and the optional-where-term rule in Smaller improvements
`%` placeholder for all-optional `` ` ``		`` ` `` groups
Explicit load of references and lists, `#Key` plus `#Repo.get(...)` when not loaded	MM § "Explizites Laden"; OF § "Beziehungen und Objektgraphen"	C4
State-transition Service Method with Status Preconditions, plus pure `...Entscheiden(x, fakten)` variant	CONV `mw-domaenenmodell_v1.md` § "Aggregate und Invarianten", § "Service und Aggregat"	B1 (generated skeleton and source-status Precondition), A2 (validations on save), B2 for document aggregates
Facts DTO for cross-aggregate rules loaded by caller and passed to the Service	CONV `mw-domaenenmodell_v1.md`; PR2 P-050	A6 (specification over several aggregates compiled to a query), B3 (eventual consistency via events instead of synchronous facts)
Validation-then-mutate block	OF § "Preconditions, Validation, Guards und Exceptions"	A2 (phase order enforced by the Session)
Edit DTO between UI and aggregate, values passed individually to the Service, input checks via `LENGTH/RANGE`	WB § "Rechnung und Positionen bearbeiten"	A1 (constraints declared once, parsed at the DTO-to-Entity boundary), D1 (property-anchored problems)
Owner plus edit Command pair with `DISABLED` form and revert of the shared root	WB § "Rechnung und Positionen bearbeiten"; CONV `mw-commands_v1.md`	Not absorbed (UI-side pattern, out of scope); B7 would add persistence of the shared Session

Hand-written pattern (what is written each time)	Where documented	Absorbed by
Termination handler merging the pushed object into `filter.results`, copying values into DTOs by hand	OF § "`cmdTermHandler`"; CONV `mw-commands_v1.md` § "Suche und Rückgabe"	C2 (re-run the view for the changed key) or B3 (handler on the Entity's event updates the result list)
Scope function loading allowed values in `command init` and binding `setScope/setElements/setEnabled`	OF § "Scopes"	B1 (enabled transitions derived from the transition table), C5 (row restrictions evaluated in the Repository)
Double-checkout avoidance via `session entities (keys of checked out)` or `#+ with`	OF § "Entities in der Session prüfen"	C4 (load graph declared once per `CHECKOUT` method avoids overlapping loads); partially remains
`isAuthenticated` body setting user name and id; also mandatory but unused on `BatchJob Module`	OF § "User Environment und User Service"; DUX § "Batchjob"	Smaller improvements (authentication provider declaration in `OFXConfig`); not a language concept family
Job user context in `OFXConfig` (`UserEnvironmentInformation`, `OFXSimpleUserServices`)	OF § "User Environment und User Service"	B6 (persisted work items carry the user and tenant context from enqueue time, as metasfresh workpackages do)
`TestDaten` Service with `CS` custom session, `session operation add`, `#CS.COMMIT()`, read-back in fresh session	CONV `mw-tests_v1.md`	C1 (graph save in one call removes most `session operation add` lines); the `__testsTestDaten` name clash (PR2 P-055) is a generator fix
Get-or-create master data in tests	CONV `mw-tests_v1.md` § "Plausible Testdaten"	C8 (`UPSERT` save variant)
Test Repository via `superclass` plus `primary instance`	OF § "Repositories im Test ersetzen"	Not absorbed; remains the mechanism for substituting external systems
Idempotent consumer Precondition re-checking that work is still needed	DUX § "Batchjob mit `BatchJob Module`"	B6 (persisted inbox with element state, `idempotent by` key)
Feature flags as `static role`	OF § "Rollen, Scopes und Identities"	Not absorbed; a small `feature flag` option would only rename the practice
Read-only user management Entity and Repository in `rollen`	CONV `mw-anwendungsaufbau_v1.md`	Not absorbed
Status-change mask forcing the Command to remember and restore the old Status around the Service check	PR2 P-042	B1 (transition declared on the Status; the UI offers transitions, not raw Status values)
Problem-log limits from the Command and Session model: `revert` of parameters only (PR1 P-014), no cancel Conclusion (PR1 P-017), `<user_cancel>` only for `SEARCH_CMD` (PR1 P-024), `GRAPH_EDIT_CMD` not top-level in tests (PR1 P-026), page-less edit returning a value only by push (PR2 P-041)	PR1, PR2	Partly B7 (persisted Session) and B4 (process steps with typed data); mostly generator and test-DSL fixes rather than new concept families
Status and null semantics: exactly one `ON_CREATION` element (PR2 P-011), filter Status starting at `ON_CREATION` (PR2 P-034), `LocalDate ==` as reference comparison (PR1 P-023), `OPTIONAL` string delegate yielding `null` (PR1 P-032)	PR1, PR2	A3 (declared nullability per property drives defaults, operators and `status switch` arms)

Hand-written pattern (what is written each time)	Where documented	Absorbed by
MySQL/MariaDB gaps: `AUTOID` sequence names ignored (PR1 P-016), `BATCH` with auto-ID not implemented (PR2 P-040)	PR1, PR2	C8 (dialect-emulated statements) and the formatted-sequence option in Smaller improvements
`CAN_OPEN_RO` locking all forms so a filtered `SEARCH_CMD` is unusable (PR2 P-060)	PR2	C5 (row restriction per role instead of a read-only Session), member-level read/write declaration in Smaller improvements

---

Smaller improvements

The following are options on existing concepts, checker rules and conveniences; each is one or two sentences with its source. They are grouped by the MoWare concept they would attach to.

Business Properties and types

- Per-operation field control: RAP's `field ( readonly : update )`, `field ( mandatory : create )` and `field ( suppress )` are combinable per-field modes per operation; as property options they would replace `#Meta setEnabled` calls in Scope functions (SAP cheat sheet 36). Note that RAP performs no runtime check for `mandatory` except `mandatory:create`, so ObjectFlow should check at check-in (SAP Learning static field control).
- Generate `status switch` with no `default` and add `case null` only when the switched property is declared nullable, so javac flags a new Status element in every unchanged switch and a `MatchException` guards the runtime (JEP 441).
- Generate Value Objects and DTOs as Java records with the constraint check in the compact constructor; records cannot extend classes, so DTO hierarchies need sealed interfaces, and JEP 401's `value record` is a one-token upgrade later (JEP 395, JEP 401).
- Emit JSpecify `@NullMarked`/`@Nullable` on generated classes once nullability is declared (JSpecify user guide).
- Lookup contract per Entity: OpenXava's `@SearchKey` (what the user types to find a reference) and Jmix's `@InstanceName` (human label, with `@DependsOnProperties`) are UI-independent declarations that DataUX and any API can reuse (OpenXava model, Jmix entities).
- Property flags from Axelor worth considering: `translatable`, `encrypted` (AES-256), `massUpdate`, `namecolumn`, `search="a,b"`, `initParam` (Axelor models).
- Three flavours of derived property, expression (recalculated live with `@Depends`), Java getter, and SQL formula, each with an explicit depends-on list; OpenXava separates them cleanly and forbids `@Depends` on SQL formulas (OpenXava model).
- Status element bound to a Service implementation (strategy dispatch), as BC's `enum ... implements Interface` with `Implementation = I = Codeunit` per value (BC interfaces).
- Money `allocate` as the one operation `BigDecimal` lacks: proportional split without losing cents (Fowler, Money; allocation detail via Verraes, secondary).
- Checker: a literal assigned to a constrained property is checked in the editor, as Iron checks `val y: Int :| Greater[0] = -1` at compile time (Iron).

Preconditions, problems and Service Methods

- Property-anchored problems, `precondition ... on rechnung.betrag`, so UI and API can place the message; Mendix `Validation Feedback(object, member, message)` and Salesforce's error location are the models (Mendix validation feedback, Trailhead validation rules).

- Collect-all mode for Preconditions in direct Service calls, returning the full problem list instead of throwing `OFXAbortedException` on the first (PR2 P-030); BC's `ErrorBehavior::Collect` and Iron's `refineValidation` are precedents (BC collecting errors).
- Prior value and change flags inside rules: Salesforce `ISNEW()`, `ISCHANGED(field)`, `PRIORVALUE(field)`, metasfresh `ifColumnsChanged`, Jmix `AttributeChanges`; a `changed(x)`/`old(x)` expression in Preconditions scopes a rule to a transition and avoids the pitfall of blocking unrelated edits on pre-existing violators (Trailhead best practices, Jmix entity events).
- Validation phase and channel tags: OpenXava's `onlyOnCreate` and `@RemoveValidator`, Odoo's `@api.ondelete` and `Many2one(ondelete=restrict|set null|cascade)`, and Jmix validation groups (`UiComponentChecks`, `RestApiChecks`) suggest tagging Preconditions with create/update/delete and UI/API/service applicability (OpenXava model, Odoo ORM, Jmix bean validation).
- Service execution policy options: OFBiz's `use-transaction`, `require-new-transaction`, `transaction-timeout`, `semaphore wait|fail` and `max-retry` name what `#+ with` custom sessions do by hand (OFBiz Service Engine Guide).
- Method semantics option `SAFE | IDEMPOTENT | NON_IDEMPOTENT`, as Causeway's `@Action(semantics=...)`, where safe methods skip the concurrency check and RAP functions "do not lock or modify data" (Causeway @Action, SAP cheat sheet 36).
- Expose a subset of an aggregate's operations per Command or API, as RAP projection BDEFs do with `use create; use action act;` (SAP cheat sheet 36).
- Checker: an `API_METHOD`'s parameter and return DTO types must themselves be published, OutSystems' exposure rule (OutSystems Server Action).
- Period control as a Precondition family keyed by document type, date, organisation and action-versus-posting (MPeriod.java).
- Re-read the Status under row lock before a lifecycle action and fail with "status changed" if another Session moved it, as iDempiere's `DocStatusChanged` (DocumentEngine.java); and an `expect status X after` assertion on transition methods, as metasfresh's `processEx` (IDocumentBL.java).
- Semantic lock: mark a Status element as `pending` so other Commands' "not while pending" Preconditions are generated (Microsoft saga).
- Decision-table expression with hit policy enum (Unique, First, Any, Rule order, Collect+SUM/MIN/MAX/COUNT) and Unique checked at test time (Camunda hit policies).

ManMap mappings and options

- Formatted business sequence: Axelor's `<sequence name initial increment padding prefix/>` assigned on save of new records, with BC's distinction between gap-free No. Series (table lock) and fast gap-tolerant SQL sequences as a `GAPLESS` flag; RAP's `early`/`late numbering` names when the key is assigned (Axelor sequences, BC number sequences, alguidelines No. Series, SAP cheat sheet 36). This also addresses the MySQL/MariaDB `AUTOID` gap (PR1 P-016).
- `OPTIMISTIC_LOCK` scope over a composition tree (RAP `total etag`) and a choice of locking columns (iDempiere `set_OptimisticLockingColumns`, OFBiz `enable-lock` requiring `lastUpdatedStamp`) (SAP cheat sheet 36, PO.java, OFBiz Entity Engine Guide).
- Pessimistic `FOR UPDATE [NOWAIT | SKIP LOCKED]` modifier on `CHECKOUT` methods, dialect-rendered by the generator (Hibernate Introduction); BC's tri-state locking and `ReadIsolation` per read show the per-statement isolation knob (BC tri-state locking).
- Checker and generator: audit values are platform-owned, creation fields are immutable and user-assigned values are overridden, as BC's system fields; `CREATEDAT`/`CREATEDBY` should not be settable from business code (BC system fields).
- Per-field change log option `TRACK` writing old/new value rows, Odoo's `tracking=True` and iDempiere's "Maintain Change Log", with BC's warning not to track high-volume ledger tables (Odoo mixins, BC change log).
- Conditional reference target by discriminator, BC's `TableRelation = if (Type = const (Customer)) Customer else if (Type = const (Item)) Item`, doubling as validation metadata (BC TableRelation).

- Read-only `EntityMapping` onto a SQL select or database view with declared base tables, Hibernate's `@Subselect`/`@Synchronize` (Hibernate @Subselect).
- Partition-key predicate added to generated UPDATE/DELETE, Hibernate `@PartitionKey` (Hibernate User Guide §3.11).
- `DEPRECATED` with Pending/Removed states checked like BC `ObsoleteState` (warning on Pending references, error on Removed) (BC ObsoleteState).
- Index hint for aggregate properties, since BC requires "appropriate SIFT indices for all FlowFields of type sum or count" (BC performance).

Repositories and queries

- Optional where-terms with correct `||` semantics: a predicate whose parameter is unset drops out, a group of only dropped predicates is neutral rather than false, and an all-optional WHERE on UPDATE/DELETE is refused, following MyBatis Dynamic SQL's `isEqualToWhenPresent` and `NonRenderingWhereClauseException` (MyBatis Dynamic SQL); this fixes PR2 P-037 and removes the `%` workaround.
- Reusable, composable filter fragments declared once in a Repository and combined with and/or in several methods, including bulk update/delete, as Spring Data Specifications (Spring Data Specifications).
- Generated AND-filter finder from a field list, Axelor's `<finder name using="firstName,lastName"/>` (Axelor repositories), and a query-by-example method with an explicit matcher, deliberately shallow (Spring Data QBE).
- `collect`/`collectChildren` navigation in mapped queries ("all business partners of matched orders", "lines of matched shipments") as a declarative join substitute without lazy loading (IQueryBuilder.java).
- `listJoin` strategy switch: JOIN versus batched IN-list versus subselect per method, the IN-list variant avoiding row multiplication on wide parents (Hibernate Introduction).
- Result-window kinds `page` (count query), `slice` (size+1 probe) and `keyset` with edit-time rules (non-null sort keys, key appended, no offset) (Spring scrolling, jOOQ seek blog).
- Checker warning for a Repository call inside a loop over Entities, making the N+1 shape visible the way governor limits force bulk patterns in Salesforce (Trailhead bulk triggers).
- Checker: a method body touching a reference not in its declared load graph, Jmix's "cannot get unfetched attribute" moved to edit time (Jmix fetching).

Batch jobs, events and processes

- Retry policy as a reusable declaration with Temporal's fields (initial interval, backoff, max interval, max attempts, non-retryable problem types, overall timeout) attachable to consumers, event handlers and process steps (Temporal retry policies).
- `INCIDENT` as a terminal reaction in the exception strategy, visible to operators and resolvable, after Camunda (Camunda job workers); dead-letter as the other place failed work can go (Axon event processors).
- Schedule options on `consumer/producer pair`: overlap policy (skip, buffer, allow), catch-up window, per-element timeout, declared "never runs concurrently" and "must be re-runnable" (Temporal schedules, ODC timers).
- `idempotent by <key>` on Commands and Service Methods with Stripe semantics (store first result, replay on retry, reject parameter mismatch, bounded retention) (Stripe idempotent requests); process-start policies `reject duplicate | use existing` (Temporal workflow id).
- Handler option `isolated` running the handler in its own transaction so its failure cannot roll back the publisher, BC's isolated events (BC isolated events).
- Event versioning: once events are persisted in a registry or outbox, old serialized events will be read by new code, so a `revision` on event types and an upcaster slot are needed; identifiers and timestamps stay immutable (Axon event versioning).
- A "not during replay" guard for side-effecting handlers if events are ever replayed, Axon's `isLive()` (Axon aggregate).

- Test DSL: `run command` asserts published events (`expect event RechnungFreigegeben with rechnungKey = ...`), Spring Modulith's `AssertablePublishedEvents` (Modulith events).
- Version binding for rule tables and forms used by running processes (`latest | deployment | versionTag`) (Camunda business rule tasks).
- Generated module canvas per Bereich (Entities, Commands, Services, events published and consumed, configuration), Spring Modulith's `writeModuleCanvases()` (Modulith documentation).

Roles, configuration and tests

- Member-level read/write per role declared once on Business Properties and derived into UI read-only state and API field omission, Mendix access rules and Causeway's `hidden(Where)`/`editing` (Mendix access rules, Causeway @Property); this is a separate topic but would resolve the `CAN_OPEN_RO` coarseness (PR2 P-060).
- Checker rules borrowed from Causeway's metamodel validator: a Command parameter of list type needs a declared choices source, and an `optional` property must agree with its `NOTNULL` mapping (Causeway validator config).
- Command-log policy `only-if-system-changed` when Commands are recorded, so safe searches are skipped (Causeway command log).
- Mixin-style contributed members: an action or derived property contributed to an Entity from another model without editing it, Causeway mixins; useful if MoWare ever needs module-external extension, otherwise low priority (Causeway mixins). Extensible enums and table extensions (BC `Extensible = true`, `enumextension`, `tableextension`) address an ISV ecosystem problem MoWare does not have and are not recommended (BC extensible enums).
- RAP strict-mode style checker rules on ObjectFlow: no `CHECKIN`/`DELETE`/`STATEMENT` inside a determination or validation, no `emit` outside `FINAL OK_CONCLUSION` or the save phase, and cycle detection over field triggers (SAP cheat sheet 08).
- Additive-only API evolution with `deprecated` flags rather than versioned endpoints, GraphQL's versionless practice (GraphQL schema design).
- Draft-style lock lifetime and expiry as configuration values if B7 is pursued: RAP uses a 15-minute exclusive lock and 28-day draft expiry, whose configurability is unverified (SAP Learning draft).